

TEAM SECURITY RUNBOOK

public-safety-alerts-aggregator

A practical, evidence-led guide for secure implementation, deployment readiness, and day-to-day operations.

Repository Azmith47/public-safety-alerts-aggregator	Prepared by Pradip	Revision date 23 August 2026
---	------------------------------	--

Purpose

Use this document as the team's shared checklist for verifying risk, implementing controls, approving a production release, and operating the public-safety alert aggregator safely.

How to Use This Runbook

This is not a penetration-test certificate. It documents architecture-led observations and the exact checks the team should run to confirm them. Treat a risk as confirmed only when its validation step produces evidence.

1. Confirm Validate the observed condition in source code and deployment configuration.	2. Fix Complete the P0/P1 controls and retain PR/test evidence.	3. Release Apply the production release gate before go-live.	4. Operate Monitor controls, rotate keys, patch dependencies, and rehearse response.
---	---	--	--

Important Next.js Clarification

Environment variables: A non-NEXT_PUBLIC_ value is server-only under standard Next.js handling. It is not automatically sent to the browser. Browser exposure occurs when a value is deliberately exposed through NEXT_PUBLIC_ or is configured using next.config.js env, which inlines configured values. A Google Maps browser key is usually public by design; protect it with HTTPS referrer and API restrictions. Keep privileged keys such as TFNSW_API_KEY server-only.

Scope and Evidence

- Reviewed paths: backend/server.js, backend/services/SchedulerService.js, backend/services/MaintenanceService.js, backend/data-collection/api_collectors/tfnswCollector.js, backend/controllers/alertsController.js, client/src/app/page.tsx, backend/config/corsOptions, and DAOs under backend/database.
- The application is an Express backend and Next.js frontend that ingests public-safety feeds, stores alert data locally, exposes API routes such as /alerts, /sources, and /geo, and renders a map interface.

Team Decisions to Record

Decision	Accountable role	Definition of done
Public API policy	Product + backend lead	Public/private endpoint list, data classification, and quota policy approved.
Maps-key classification	Frontend + cloud owner	Browser key referrer/API restrictions verified; privileged keys server-only.
Production trust boundaries	DevOps + backend lead	Feed allowlist, database access, backups, service accounts, and egress rules documented.
Security release owner	Engineering lead	Named approver for staging sign-off, production release, and incident escalation.

Current Condition: Verify, Then Act

The following cards deliberately separate what was observed from what must be checked in the live repository and environment.

Secrets and client configuration

Observed GOOGLE_MAPS_API_KEY is referenced in client/src/app/page.tsx; collectors use TFNSW_API_KEY.

Validate Inspect next.config.js, component boundaries, generated build artifacts, and Git history.

Required action Keep TFNSW_API_KEY server-only. Restrict any browser Maps key by HTTPS referrer and API scope; rotate any exposed privileged key.

External data and stored XSS

Observed Collectors transform upstream content that is later stored and rendered.

Validate Find dangerouslySetInnerHTML and any raw HTML paths. Use a safe staging fixture containing a script payload.

Required action Schema-validate input, sanitize rich text or store plain text, and avoid raw HTML rendering unless it is sanitized.

SQL construction

Observed MaintenanceService creates SQL fragments; DAOs may accept condition strings.

Validate Trace whether request, feed, or environment values can alter SQL syntax.

Required action Use bound query parameters only; parse retention values as bounded integers; never accept arbitrary condition strings.

API abuse resistance

Observed Public client routes are present; rate limiting is not assumed.

Validate Confirm public/private endpoint policy, real auth behavior, payload limits, and rate-limit responses.

Required action Add route limits, payload limits, caching where appropriate, and auth for administrative/write operations.

Collectors and schedulers

Observed Background jobs fetch upstream sources and perform maintenance.

Validate Check timeout, redirects, concurrency, retries, overlap, and persistent failure reporting.

Required action Use fixed URL allowlists, AbortController timeouts, low concurrency, backoff with jitter, and an overlapping-run lock.

CORS, TLS, and errors

Observed cors(corsOptions) and local HTTP development references exist.

Validate Review real CORS config and reverse-proxy TLS/error behavior.

Required action Allow only trusted origins, enforce TLS/HSTS at edge, and keep stack traces out of production responses.

Dependencies and telemetry

Observed Lockfiles and logger middleware are present.

Validate Run npm audit; inspect CI, logging retention, dashboard, and alert configuration.

Required action Use dependency scanning, structured redacted logs, metrics, alerts, and tested response procedures.

Priority Plan

Priority	Focus	Release evidence
P0	Secret review; validation/sanitization; SQL parameterization.	No exposed privileged secrets; XSS and SQL tests pass.
P1	CORS/TLS, rate limiting, collector resilience, secure errors.	Staging verifies origin policy, TLS, limits, and scheduler behavior.
P2	Dependency automation, centralized observability, restore capability.	Alerts work and a backup restoration is recorded.
P3	WAF/API gateway, SBOM, signing, OIDC/SIEM.	Tracked roadmap with owner and target date.

Pre-Deployment Roadmap

P0 — Before merging to main

- Create an environment-variable inventory: owner, classification (public browser/server secret), storage location, permitted scope, and rotation date.
- Add AJV schemas per collector. Reject malformed records and cap text fields, request bodies, and upstream response sizes.
- Sanitize external free text before persistence where HTML is not required. If markup is necessary, permit a minimal allowlist and sanitize before raw rendering.
- Replace dynamic SQL values with parameterized queries. Use fixed internal fragments only after bounded parsing of configuration values.
- Install Helmet, restrictive CORS, API rate limiting, and body-size limits. Confirm generic production error handling.
- Add secret scanning to pre-commit and CI. If a secret is exposed, revoke and rotate it; deleting it from the latest commit is insufficient.

P1 — Staging readiness

- Use fixed source allowlists, network timeouts, redirect limits, concurrency caps, exponential backoff with jitter, and locks preventing overlapping scheduler runs.
- Run CI stages: lint → tests → SAST → dependency scan → build → staging → DAST → approval → production.
- Add stored-XSS fixtures, SQL parameterization checks, CORS allow/deny tests, rate-limit checks, protected-route auth tests, and restore tests.
- Enforce TLS at the load balancer/reverse proxy; enable HSTS after confirming stable HTTPS. Use least-privilege identities, restricted database permissions, encrypted backups, and a secret manager.

Production Release Gate

Gate	Pass condition
P0 completion	All P0 tests pass; no unresolved critical/high issue without documented risk acceptance.
Secrets	Privileged secrets absent from repository/build artifacts or revoked and rotated.
Exposure	CORS is restricted, TLS works, and production errors expose no stack traces.
Resilience	Rate/payload limits and scheduler safeguards verified in staging.
Recoverability	Alerts route correctly and a backup restore test has a recorded result.

Operational Controls

Control	Measure	Cadence / trigger
Application health	5xx rate, latency, availability, rate-limit rejections.	Alert on sustained threshold breach; daily review.

Ingestion health	Per-source success, volume, rejects, duration, retries, last success.	Alert on missed runs, failure bursts, anomalous volume, validation failures.
Data and backup	DB growth, backup completion, encryption/access, restore health.	Daily status; quarterly restore exercise.
Vulnerability management	Dependency/container scan and patch status.	Every PR; critical issues immediately; updates monthly.
Keys and access	Credential inventory, scope, rotation, and usage.	Provider policy and immediately after suspected exposure.

Implementation Reference

Express baseline — adapt to existing backend/server.js

```
const helmet = require('helmet') const rateLimit
= require('express-rate-limit') const cors =
require('cors')

const allowedOrigins = new Set( (process.env.FRONTEND_ORIGINS ||
'').split(',').map(x => x.trim()).filter(Boolean) ) app.use(helmet())
app.use(express.json({ limit: '100kb' })) app.use(cors({ origin(origin, cb) {      if
(!origin || allowedOrigins.has(origin)) return cb(null, true)      return cb(new
Error('Origin not allowed'))    } }))) const apiLimiter = rateLimit({ windowMs: 15 *
60 * 1000, limit: Number.parseInt(process.env.RATE_LIMIT_MAX || '100', 10),
standardHeaders: 'draft-8', legacyHeaders: false })
app.use(['/alerts', '/sources', '/geo'], apiLimiter)
```

Implementation notes

- Use credentials: true only when the browser genuinely makes credentialed cross-origin requests.
- In production, validate FRONTEND_ORIGINS at startup and fail closed if no approved origin is configured.
- Apply stricter limits to expensive endpoints and use an external/shared rate-limit store when scaling beyond oneprocess.

Collector validation and sanitization

```
const sanitizeHtml = require('sanitize-html')
const Ajv = require('ajv') const ajv = new
Ajv({ allErrors: true })

const validateAlert = ajv.compile({ type: 'object',
additionalProperties: false, required: ['id', 'title',
'pubDate'], properties: { id: { type: 'string',
maxLength: 128 }, title: { type: 'string', minLength: 1,
maxLength: 300 }, description: { type: 'string',
maxLength: 10000 }, link: { type: 'string', maxLength:
2048 }, pubDate: { type: 'string' } } }) function
sanitizeText(value = '') {
  return sanitizeHtml(String(value), { allowedTags: [], allowedAttributes: {} }).trim() }
function validateAndNormalizeAlert(raw) {
  const alert = { ...raw, title: sanitizeText(raw.title),
description: sanitizeText(raw.description) || '' }
  if (!validateAlert(alert)) throw new Error(JSON.stringify(validateAlert.errors))
  return alert }
```

Use validation immediately before persistence. Validate links separately with new URL(), permit only http/https, and use source-specific schemas when source structures differ.

SQL parameterization

```
const days = Number.parseInt(process.env.ALERT_RETENTION_DAYS || '30', 10) if
(!Number.isInteger(days) || days < 1 || days > 3650) throw new Error('Invalid retention')
const threshold = new Date(Date.now() - days * 86400000).toISOString() db.prepare('DELETE FROM
alerts WHERE created_at < ?').run(threshold) // Avoid: db.exec('DELETE FROM alerts WHERE
created_at < ${value}')
```

Testing and CI Baseline

- On every pull request: secret scan, lint, unit/integration tests, Semgrep/SAST, and dependency scan.
- Test XSS with a script-tag fixture; test DAOs with SQL metacharacters; test CORS, rate limits, authentication, and scheduler overlap.
- Deploy to isolated staging with test data; run DAST under explicit approval.
- Require a release owner's sign-off for changes affecting exposure, authentication, collectors, secrets, databaseschema, or CI/CD.

Incident Response: First Actions

- Detect and scope the alert; preserve logs and relevant metadata.
- Contain by disabling the affected collector, revoking credentials, and tightening WAF/rate-limit controls.
- Investigate using a database snapshot and recent commit/deployment history.

- Patch, rotate, restore verified clean data, and only re-enable after health checks.
- Document timeline, impact, root cause, remediation, and follow-up ownership.

Document Control

- Review after architecture changes, new data sources, new public endpoints, incidents, or at least every six months.
- For each completed gate, record the PR, test result, reviewer, date, and any risk acceptance.